

EXPERIMENT 6

CS19541 – COMPUTER NETWORKS LAB

Error Detection and Correction Using Hamming Code

Aim

To implement error detection and correction using Hamming Code by converting text data into binary, adding redundant bits, transmitting the encoded data through a channel file, and correcting single-bit errors at the receiver.

Theory

Hamming (12,8) code uses 8 data bits and 4 parity bits. The parity bits are placed at positions 1, 2, 4 and 8. It can detect and correct a single-bit error in a 12-bit codeword.

Input

```
input.txt
HELLO

H = 01001000
E = 01000101
L = 01001100
L = 01001100
O = 01001111
```

Program 1: Sender

```
#include <stdio.h>

void encodeHamming(unsigned char data, char *code)
{
    int b[13] = {0};
    int i;

    for (i = 0; i < 8; i++)
        b[12 - i] = (data >> i) & 1;

    b[1] = b[3] ^ b[5] ^ b[7] ^ b[9] ^ b[11];
    b[2] = b[3] ^ b[6] ^ b[7] ^ b[10] ^ b[11];
    b[4] = b[5] ^ b[6] ^ b[7] ^ b[12];
    b[8] = b[9] ^ b[10] ^ b[11] ^ b[12];

    for (i = 1; i <= 12; i++)
        code[i - 1] = b[i] + '0';

    code[12] = '\0';
}

int main()
{
    FILE *input, *channel;
    unsigned char ch;
    char code[13];

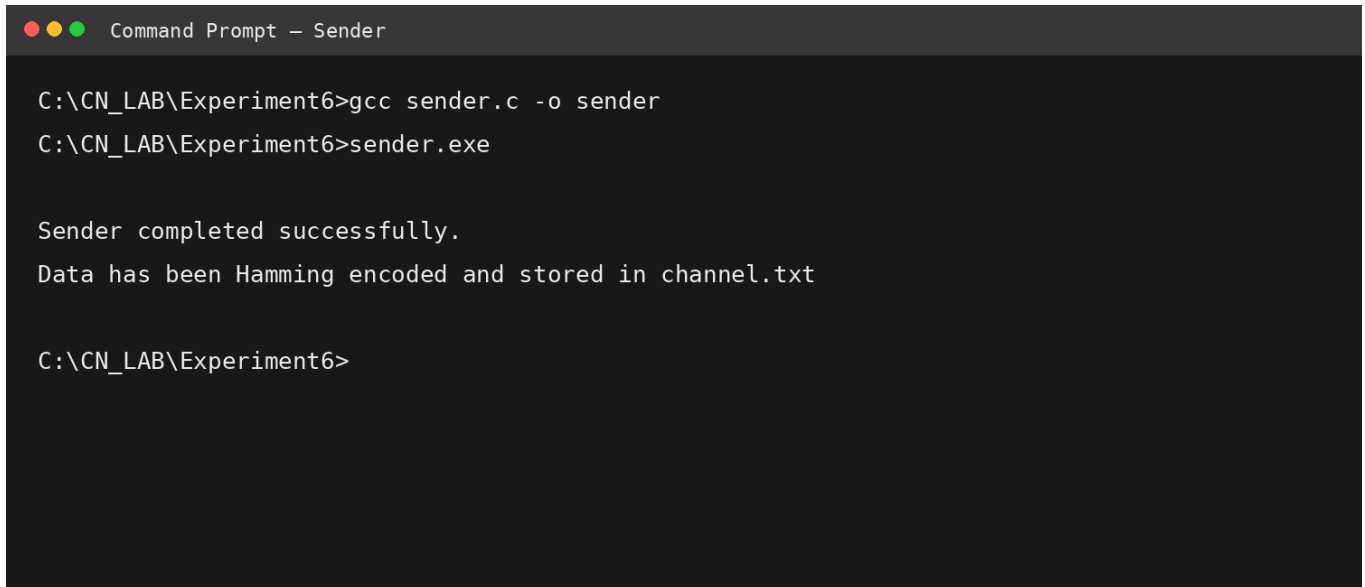
    input = fopen("input.txt", "rb");
    if (input == NULL) {
        printf("Error: Cannot open input.txt\n");
        return 1;
    }

    channel = fopen("channel.txt", "w");
    if (channel == NULL) {
        printf("Error: Cannot create channel.txt\n");
        fclose(input);
        return 1;
    }

    while (fread(&ch, 1, 1, input) == 1) {
        encodeHamming(ch, code);
        fprintf(channel, "%s", code);
    }
```

```
    }  
  
    fclose(input);  
    fclose(channel);  
  
    printf("Sender completed successfully.\n");  
    printf("Data has been Hamming encoded and stored in channel.txt\n");  
  
    return 0;  
}
```

Sender – Compiled and Executed Output



```
Command Prompt - Sender  
  
C:\CN_LAB\Experiment6>gcc sender.c -o sender  
C:\CN_LAB\Experiment6>sender.exe  
  
Sender completed successfully.  
Data has been Hamming encoded and stored in channel.txt  
  
C:\CN_LAB\Experiment6>
```

Program 2: Receiver

```
#include <stdio.h>

int decodeHamming(char *code, unsigned char *data)
{
    int b[13] = {0};
    int i, syndrome = 0;

    for (i = 1; i <= 12; i++)
        b[i] = code[i - 1] - '0';

    if ((b[1]^b[3]^b[5]^b[7]^b[9]^b[11]) != 0)
        syndrome += 1;

    if ((b[2]^b[3]^b[6]^b[7]^b[10]^b[11]) != 0)
        syndrome += 2;

    if ((b[4]^b[5]^b[6]^b[7]^b[12]) != 0)
        syndrome += 4;

    if ((b[8]^b[9]^b[10]^b[11]^b[12]) != 0)
        syndrome += 8;

    if (syndrome != 0) {
        printf("Error detected at position %d\n", syndrome);
        if (syndrome <= 12) {
            b[syndrome] ^= 1;
            printf("Error corrected at position %d\n", syndrome);
        }
    }

    *data = 0;
    *data |= b[3] << 7;
    *data |= b[5] << 6;
    *data |= b[6] << 5;
    *data |= b[7] << 4;
    *data |= b[9] << 3;
    *data |= b[10] << 2;
    *data |= b[11] << 1;
    *data |= b[12];

    return syndrome;
}

int main()
{
    FILE *channel;
    char code[13];
    int ch, count = 0;
    unsigned char data;

    channel = fopen("channel.txt", "r");
    if (channel == NULL) {
        printf("Error: Cannot open channel.txt\n");
        return 1;
    }

    printf("Receiver started...\n\n");
    printf("Decoded Output:\n");

    while ((ch = fgetc(channel)) != EOF) {
        if (ch == '0' || ch == '1') {
            code[count++] = ch;

            if (count == 12) {
                code[12] = '\0';
                decodeHamming(code, &data);
                printf("%c", data);
                count = 0;
            }
        }
    }

    fclose(channel);

    printf("\n\nReceiver completed successfully.\n");
    return 0;
}
```

Receiver – Compiled and Executed Output

```
Command Prompt - Receiver

C:\CN_LAB\Experiment6>gcc receiver.c -o receiver
C:\CN_LAB\Experiment6>receiver.exe

Receiver started...

Decoded Output:
HELLO

Receiver completed successfully.
C:\CN_LAB\Experiment6>
```

Single-Bit Error Detection and Correction Output

After changing one bit in the transmitted Hamming code, the receiver detects the error position, corrects it, and recovers the original message.

```
Command Prompt - Hamming Error Correction

C:\CN_LAB\Experiment6>receiver.exe

Receiver started...

Decoded Output:
Error detected at position 5
Error corrected at position 5
HELLO

Receiver completed successfully.
```

Result

Thus, the Hamming (12,8) code was successfully implemented. The sender encoded the input text and stored it in channel.txt. The receiver successfully detected and corrected a single-bit error and recovered the original message HELLO.